

SOLIDWORKS API Help

Overview, Navigation, Syntax Conventions, Resources, and Examples

Converted from apihelp.chm



Welcome to API Help

You can use a software product's API (Application Programming Interface) to:

- automate and customize the product.
- write applications that add functionality to the product, typically referred to as add-in applications.

Each product's API contains multiple functions that you can call from Visual Basic for Applications (VBA), VB.NET, Visual C#, Visual C++ 6.0 , and Visual C++/CLI. These functions provide direct access to product's functionality.

API Help is available on the Web and locally. See **Accessing API Help** in [Access and Navigation](#).

Use the **Send comments** link located in the upper-right corner of a reference topic to report errors and omissions in the Help. The subject line of the mail message sent to **API Help Feedback** will automatically contain the name of the reference topic that you are reading. If you have feedback about a topic without a **Send Comments** link, then send your feedback to apihelp.feedback@3ds.com. Your feedback is important because it will help Dassault Systèmes SolidWorks Corporation improve the Help, which will hopefully benefit you. Please continue to send functionality related questions about any of the APIs to [SOLIDWORKS API Support](#).

NOTE: All of the sample code supplied is provided on an as-is basis, and is only intended to demonstrate ways of using an API. No representations or warranties are made regarding these samples. Any licensed user of an API is free to use any or all of these samples in connection with building applications related to that API, and is granted a royalty-free, non-exclusive license for these samples, or parts thereof. Intellectual property rights of the samples remain with SOLIDWORKS. Any confidentiality provisions of the applicable product license apply to the samples.

Access and Navigation

Accessing API Help

Help is available on the Web or in your local installation directory. To enable your access to API Web Help, select SOLIDWORKS menu ? > **Use Web Help**. A check mark indicates that API Help is enabled and accessible on the Web.

To open context-sensitive SOLIDWORKS API Help from:

1. SOLIDWORKS VB.NET or C# macros opened in Visual Studio 2015:

1. In the IDE, select **Help > Set Help Preference > Launch in Help Viewer**.
2. Put your cursor on the interface, method, property, or delegate whose help topic you want displayed and press the F1 key.

The associated help topic is displayed in Microsoft Help Viewer 2.2.

2. SOLIDWORKS VB.NET or C# applications opened in Visual Studio 2012:

1. In the IDE, put your cursor on the interface, method, property, or delegate whose help topic you want displayed and press the F1 key.
2. If the associated help topic is not displayed, then select **Tools > Options** in the Help Viewer.

1. Select **Online**.
2. Select **Try local first, then online** or **Try local only, not online**.
3. Select **Local Help**.
4. Click **OK**.
5. Put your cursor on the interface, method, property, or delegate in the IDE again and press F1.

The associated help topic is displayed.

3. SOLIDWORKS VBA macros:

In the SOLIDWORKS Microsoft Visual Basic Integrated Developer Environment (IDE), put your cursor on the interface, method, property, or event whose help topic you want displayed and press the F1 key. The associated help topic is displayed.

IMPORTANT NAVIGATION CHANGES IN API HELP IN 2009 AND LATER

- Reference topics now use the name of the interface, not the object, as in earlier API Help systems.

So, in this Help, it is:

- IAdvancedSelectionCriteria **not** AdvancedSelectionCriteria,
- IAnnotation **not** Annotation,
- IAnnotationView **not** AnnotationView,
- IAssemblyDoc **not** AssemblyDoc, etc.

All interface, method, properties, and delegate topics contain links to their corresponding Visual Basic for Applications (VBA) object, method, properties, and event topics. Additionally each Visual Basic for Applications (VBA) object topic includes that object's object model in graphical format, if an object model exists. If an object model does not exist, then the object's topic is blank.

- The SOLIDWORKS interfaces are no longer categorized by functionality on the Contents tab. Instead, they appear in alphabetical order in their respective interop assembly namespace book. If you prefer to locate SOLIDWORKS reference (i.e., interface, method, property, and delegate topics) topics categorized by functionality, then open the [Functional Categories](#) topic in the SOLIDWORKS APIs book, which is located in the SOLIDWORKS API Help book. The SolidWorks.Interop.sldworks Namespace book contains the majority of the reference topics for the SOLIDWORKS interfaces.
- Topics for obsolete interfaces, methods, properties, and delegates are not accessible from the Contents tab; however, they are accessible using the Index and Search tabs. *Always read the description at the top of a topic.* A link to the current version of a topic appears in the description. Descriptions for obsolete topics have intentionally been omitted.

Navigating API Help

NOTE: The following information applies to API Help when accessed locally only.

- **Index and Contents Tabs**

If you are looking for specific functionality, but do not know the name of the function (interface, method, property, delegate, or enumerator), the quickest way to locate the function is to use the Index tab.

Index tab

Click the Index tab and type in the first few letters of the function whose topic you want displayed. For example to find the assembly interface topic, you can type **assemb** and **Assemblies (see also IAssemblyDoc Interface)** is selected. Double-click **Assemblies (see also IAssemblyDoc Interface)**. Select **IAssemblyDoc Interface** and click **Display** in the Topics Found dialog.

Contents tab

You can also peruse the table of contents. Click the Contents tab, click a book to expand it, and examine its topics. API reference topics appear by product in their interop assembly namespace book or books.

All reference (interface, method, property, and delegate) topics include breadcrumbs near the top of the topic. These breadcrumbs can help you determine which product's topic you are reading. Additionally the name of a method's or property's interface appears enclosed in parentheses after the name of the method or property at the top of the topic and at the end of the breadcrumb.

- **Search Tab**

If you cannot find the function that you want using the Index or Contents tabs, you can use the Search tab.

Type a word or phrase on the Search tab to see a list of topics that contain that word or phrase. For example, type **tangent edges** and click **List Topics** on the Search tab to see the topics that contain that exact or similar phrase. To reduce the number of topics found, clear the **Match similar words** (if selected) and select the **Search titles only** (if not selected) check boxes, which appear after the list. By default, the search results are ranked by number of occurrences of the word or phrase in the topic. You can alphabetize the search results by clicking **Title**.

You can also use wildcard expressions and boolean operations in your search word or phrase.

NOTE: If you specify an object by name in your search (e.g., Annotation), then the interface topic will appear in the search results (e.g., IAnnotation), not the object topic. Object topics do not appear in the search results; only the interface topics.

Wildcard Expressions (*, ?)

You can type the exact name of the function or you can use an * (asterisk) in your search expression to locate all of the topics that contain the search expression. For example, type **noti*** to locate all of the topics containing the word **notify**, **notifies**, **notification**, **notice**, and so on.

To locate all of the topics that contain just the word **notify**, type **notif?**. The ? (question mark) indicates to search for the specified expression string plus one character.

NOTE: The * and ? characters can appear at the beginning, embedded within, or at the end of a search expression. However, they cannot be the only character in the search expression.

Boolean Operators (AND, NEAR, NOT, OR)

You can use Boolean operators in the search expression. For example, if you are trying to find the SOLIDWORKS method that adds a mate condition between two assembly components, click the Search tab and type the following keywords and the Boolean operator **AND**:

assembly AND mate AND relationship

This search expression locates the **IAssemblyDoc Interface Members**, **IAssemblyDoc Interface Methods**, **AddMate3 Method (IAssemblyDoc)**, **EditMate2 Method (IAssemblyDoc)**, and, of course, this topic.

Other Boolean operators that you can use in your search string are:

- **NEAR** – Locate all of the topics that contain the specified terms, close together.
- **NOT** – Locate all of the topics that contain the term specified before **NOT**, then exclude all of the topics that contain the term specified after **NOT**
- **OR** – Locate all of the topics that contain any of the specified terms.

Syntax Conventions

[See Also](#)

This topic explains the syntax conventions used with the API Help:

- [Notation](#)
- [Methods and arguments](#)
- [Return values](#)
- [Interfaces](#)
- [Properties](#)
- [Availability links](#)

Notation

All SOLIDWORKS and add-in API functions support the COM interface. If you are using COM, the SOLIDWORKS or add-in API function returns an [HRESULT, and any additional return values](#) are passed by reference as arguments.

[Back to top](#)

Methods and arguments

The following syntax uses the SOLIDWORKS method, [IModelDocExtension::SelectByID2](#), which is a function that returns a BOOLEAN value. It passes nine arguments to the function.

Visual Basic .NET	<pre>Dim boolstatus As Boolean = false boolstatus = swModelExtension.SelectByID2("Point1", "SKETCHPPPOINT", .2, .3, 0, False, 0, Nothing, swSelectOptionDefault)</pre>
Visual Basic for Applications (VBA):	<pre>Dim Result as Boolean Result = swModelExtension.SelectByID2("Point1", "SKETCHPOINT", .2, .3, 0, False, 0, Nothing, swSelectOptionDefault)</pre>
C#	<pre>bool boolstatus = false; boolstatus = swModelExtension.SelectByID2("Point1", "SKETCHPOINT", .2, .3, 0, false, 0, null, swSelectOptionDefault);</pre>
Unmanaged C++ Dispatch:	<pre>BOOL Result = FALSE; Result = swModelExtension.SelectByID2(_T("Point1"),_T("SKETCHPOINT"), .2, .3, 0,0,0,0,swSelectOptionDefault);</pre>
Unmanaged C++ COM:	<pre>VARIANT_BOOL Result = FALSE; HRESULT hres = swModelExtension- >SelectByID2(_T("Point1"),_T("SKETCHPOINT"), .2, .3, 0,0,0,0,swSelectOptionDefault,&result);</pre>
C++/CLI	<pre>bool bSelected = swModelExtension->SelectByID2(swBody->Name, "SOLIDBODY", 0.0, 0.0, 0.0, true, 0, nullptr, (int)swSelectOption_e::swSelectOptionDefault);</pre>

NOTE: The return value in the **Visual Basic (Usage)** section in the Help is always shown as **value**. Because some methods and properties contain an argument named **Value**, SOLIDWORKS recommends that you rename the return value variable **value** to something meaningful to avoid an error in your code and for clarity.

The next example uses the SOLIDWORKS method, [IDrawingDoc::EditSheet](#), which is a subroutine which returns nothing (**void**). The method accepts no arguments.

Visual Basic .NET	<code>swDrawing.EditSheet()</code>
Visual Basic for Applications (VBA):	<code>swDrawing.EditSheet</code> (see the Note below this table)
C#	<code>swDrawing.EditSheet();</code>
Unmanaged C++ Dispatch:	<code>swDrawing.EditSheet();</code>
Unmanaged C++ COM:	<code>HRESULT hres = swDrawing->EditSheet();</code>
C++/CLI	<code>swDrawing->EditSheet();</code>

NOTE: Some methods are subroutines (that return nothing (**void**)) that also pass arguments. The VBA syntax for those methods is slightly different, despite what the Visual Basic for Applications (VBA) Syntax section of the help indicates. The help shows parentheses in the method signatures of all VBA methods. In practice, parentheses are not used in the VBA method signature of subroutines that return nothing. For example, `IModelDoc2::SetUnits` is a subroutine that returns nothing but passes arguments. The correct way to call `IModelDoc2::SetUnits` in VBA:

```
Part.SetUnits swINCHES, swFRACTION, 16, 0, False
```

[Back to top](#)

Return values

The following syntax uses the SOLIDWORKS method, [IModelDoc2::GetType](#), which is a subroutine that returns a long value. This function takes no arguments. Instead, it uses the current [IModelDoc2](#) object and returns its type.

Visual Basic .NET	<code>int docType = 0;</code> <code>docType = swModel.GetType();</code>
Visual Basic for Applications (VBA):	<code>Dim docType As Long</code> <code>docType = swModel.GetType</code>
C#	<code>int docType = 0;</code> <code>docType = swModel.GetType();</code>
Unmanaged C++ Dispatch:	<code>long docType = 0;</code> <code>docType = swModel .GetType();</code>
Unmanaged C++ COM:	<code>long docType = 0;</code> <code>HRESULT hres = swModel ->GetType(&docType);</code>
C++/CLI	<code>swDocumentTypes_e nDocumentType;</code> <code>nDocumentType = (swDocumentTypes_e)swModel->GetType();</code>

[Back to top](#)

Interfaces

Some functions have different versions of syntax. A function has multiple interfaces only when required. Always check the **See Also** link near the bottom of topic to see if a function has multiple interfaces (e.g., [IBody2::GetFirstFace](#) and [IBody2::!GetFirstFace](#)).

Visual Basic .NET	Dim swFace as Face2 = Nothing swFace = swBody.GetFirstFace
Visual Basic for Applications (VBA):	Dim swFace As Object Set swFace = swBody.GetFirstFace
C#	Face2 swFace = null; swFace = (Face2) swBody.GetFirstFace();
Unmanaged C++ Dispatch:	IDispatch* faceDisp = NULL; faceDisp = swBody.GetFirstFace(); IFace2 swFace(faceDisp);
Unmanaged C++ COM:	LPFACE2 swFace = NULL; HRESULT hres = swBody->IGetFirstFace(&swFace);
C++/CLI	IFace2^ swFace = nullptr; swFace = (IFace2^)swBody->GetFirstFace(); - or - IFace2^ swFace = nullptr; swFace = swBody->IGetFirstFace();

The Dispatch interface returns a Dispatch pointer, while the COM interface returns an LPFACE2 pointer. The COM interface uses object pointers instead of Dispatch pointers and pointers instead of VARIANT SafeArrays. Because the argument types are different, any API function that handles objects or arrays has two distinct interfaces, one for COM and one for Dispatch.

[Back to top](#)

Properties

The following syntax uses the SOLIDWORKS property [IFeature::Name](#) to get and set the name of a feature.

Visual Basic .NET	get: featName = swFeat.Name set: swFeat.Name = featName
Visual Basic for Applications (VBA):	get: featName = swFeat.Name set: swFeat.Name = featName
C#	get: featName = swFeat.Name; set: swFeat.Name = featName;
Unmanaged C++ Dispatch:	get: featName = swFeat.GetName(); set: swFeat.SetName(featName);
Unmanaged C++ COM:	get: hres = swFeat->get_Name(&featName); set: hres = swFeat->put_Name(featName);
C++/CLI	get: featName = swFeat->Name; set: swFeat->Name = featName;

[Back to top](#)

Availability links

The **Availability** links appearing at the bottom of a method, property, and delegate topic identifies the release in which the method, property, or delegate was introduced. Topics in the API Help lacking **Availability** links indicate that the method, property, or delegate was introduced prior to the SOLIDWORKS 98Plus release.

[Back to top](#)

Obsolete APIs

APIs are continually enhanced. In most instances, when an existing API is superseded by a new API, the new API has the same name as the existing API followed by a number. A higher number indicates a more recent API. For example, the method **IEntity::Select4** superseded **IEntity::Select3**, which superseded **IEntity::Select2**, etc. In this example, **IEntity::Select3**, **IEntity::Select2** and **IEntity::Select** are obsolete APIs.

It is recommended that you use the newest APIs in your code, and, whenever possible, update your existing code to use the newest APIs.

Topics for obsolete interfaces, methods, properties, and delegates are not accessible from the Contents tab; however, they are accessible from the Index and Search tabs. *Always read the description at the top of a topic.* A link to the current version of a topic appears in the description. Descriptions for obsolete topics published prior to the SOLIDWORKS 2009 SP0 release have intentionally been omitted.

Additional resources for product-specific information are:

SOLIDWORKS Design

Start SOLIDWORKS Design and click **Help > SOLIDWORKS Help**.

SOLIDWORKS Costing

Start SOLIDWORKS, click **Help > SOLIDWORKS Help**, and open the **SOLIDWORKS Costing** book.

SOLIDWORKS PDM Professional

Start SOLIDWORKS PDM Professional Administration tool, log into a SOLIDWORKS PDM vault as **admin**, expand the vault, right-click **Add-ins**, and click **Programmer's Reference Guide**.

FeatureWorks

Start SOLIDWORKS, click **Help > SOLIDWORKS Help**, expand the **Features** book, and open the **FeatureWorks** book.

eDrawings

Start the eDrawings software and click **Help > eDrawings Help Topics**.

SOLIDWORKS Routing

Start SOLIDWORKS, click **Help > SOLIDWORKS Help**, and open the **Routing** book.

SOLIDWORKS Simulation

Start SOLIDWORKS, click **Help > SOLIDWORKS Help**, and open the **Simulation** book.

SOLIDWORKS Sustainability

Start SOLIDWORKS, click **Help > SOLIDWORKS Help**, and expand the **Sustainability Products** book, and open the **SOLIDWORKS Sustainability** book.

SOLIDWORKS Toolbox

Start SOLIDWORKS, click **Help > SOLIDWORKS Help**, and open the **Toolbox** book.

SOLIDWORKS Utilities

Start SOLIDWORKS, click **Help > SOLIDWORKS Help**, and open the **SOLIDWORKS Utilities** book.

Contacting SOLIDWORKS API Support

It is our goal to provide the best support possible for the APIs. Understanding how this support works, and the type of information we need to assist you, can help us solve your problems as quickly and efficiently as possible.

API Support

An API is a thin layer over the user interface. Thus, if there is a problem in the API, it often exists in the user interface. If you can reproduce the problem in the user interface, report the problem to the user-interface documentation team. Feedback links are at the top of user-interface documentation pages.

When the problem is fixed in the user interface, then the fix is automatically applied to the API.

If the problem is only in the API, then you need to create a service request in the 3D Support Application. Please read the relevant introduction page:

For Customers: <https://www.3ds.com/support/discover-3dsupport-app/discover-3dsupport-app-for-customers/>

For VARs: <https://www.3ds.com/support/discover-3dsupport-app/discover-3dsupport-app-for-partners/>

Information to include in your service request

- Your [contact information](#).
- Your product serial or license number.
- A description of your problem. Please create separate service requests for each problem to ensure appropriate tracking.
- All API methods, properties, and events related to your problem.
- Product version, including service pack. If your problem is a regression, report the last version in which your code worked correctly, including the service pack number.
- All [test files](#) (SOLIDWORKS documents and so on).
- Graphics driver details (if the problem is graphics related).
- Operating-system details (if you suspect the problem is operating-system related).
- Source code for your project as per these [guidelines](#).

NOTE: Whenever possible, avoid using [obsolete APIs](#) in your code; instead, update your code to use the most recent APIs to see if that fixes your problem.

SOLIDWORKS API Examples, Sample Projects, and Tips

To locate SOLIDWORKS examples in the Help, use the index tab.

Examples are grouped by language on the Index tab. Type:

- **C# examples** to see available C# examples.
- **C++/CLI examples** to see available managed C++ examples.
- **Visual Basic .NET (VB.NET) examples** to see available VB.NET examples.
- **VBA examples** to see available Visual Basic for Applications (VBA) examples.
- **C++ examples** to see available unmanaged C++ examples.

Examples are also linked to relevant reference topics. See the **Example** section in a reference topic.

The **SOLIDWORKS API Support** web page, which is part of the [SOLIDWORKS website](#), provides you access to additional API examples, sample projects, tips, discussion forum, knowledge base, etc.

NOTE: You must be connected to the internet to view the **SOLIDWORKS API Support** web page and SOLIDWORKS website.

eDrawings API Sample Project

The eDrawings API sample projects show how to create eDrawings API applications in Microsoft Visual Studio and other types of applications that support ActiveX controls. You can obtain the eDrawings API SDK, which contains the eDrawings API sample projects, from [SOLIDWORKS API Support](#).

NOTES:

- All of the sample code and projects are provided on an as-is basis and are only intended to demonstrate ways of using the eDrawings API. No representations or warranties are made regarding these samples. Any licensed user of eDrawings is free to use any or all of these samples in connection with building applications related to eDrawings and is granted a royalty-free, non-exclusive license for these samples or parts thereof. Intellectual property rights of the samples remain with eDrawings. Any confidentiality provisions of the eDrawings license apply to these samples.